

Projektmunka I.

Garai Maja

Számítógépes hangfeldolgozás

Riasztó panel hangfeldolgozó szoftverrel (POD)

2023

Kandó Kálmán Villamosmérnöki kar HTT

Tartalomjegyzék

1. Projektleírás.....	3.
2. Hardver bemutatása.....	4.
3. Szoftver részegységeinek bemutatása.....	5.
3.1. Mikrovezérlő program.....	5.
3.1.1. Soros kommunikáció.....	5.
3.1.2. Hangérzékelő egység.....	6.
3.1.3. Egyéb kódrészek.....	6.
3.2. Számítógépes program.....	7.
3.2.1. Kezelői felület.....	7.
3.2.2. Alkalmazás működése.....	9.
3.2.3. Beérkező hangadat feldolgozása.....	10.
4. Továbbfejlesztési lehetőségek.....	11.
5. Végző.....	12.

1. Projektleírás

A projektem témája, egy mikrovezérlő alapú hangfigyelő rendszer amely egy hardveres és egy szoftveres komponensből áll. A lényege, hogy ezen komponensek kommunikálnak egymással, egy egészet alkotnak. A megvalósításhoz tehát mind szerelésre, forrasztásra, mind programozásra is szükség volt. A projekt jelen formájában még csak kezdetleges, de működőképes és remekül prezentálja a feladatot. Továbbfejlesztési lehetőségeiről a későbbiekben szó lesz.

A projekt jelen formájában egy hang monitorozó/riasztó egységként funkcionál, és mivel ezen adatokat egy számítógép segítségével vizualizáljuk, dolgozzuk fel, így a projekt kimeríti a számítógépes hangfeldolgozás témáját. A projekt hardveres megvalósításához Atmel mikrokontrollert alkalmaztam, amely egy Arduino környezetbe van integrálva. Ez egy rendkívül jól fejleszthető dinamikus fejlesztői környezet, amely kiváló táptalaj bármilyen demó projekt számára. A hardveren elhelyeztem visszajelző ledek, magát a hangérzékelő szenzort és a kivezetéseket, melyeken keresztül a mikrovezérlő portjaira vannak kötve. (a hardverről még később) A szoftveres megvalósítás pedig C# és XAML nyelven történt a VisualStudio nevű programban, illetve mikrovezérlő szoftveres része C-ben, az Arduino saját fejlesztői környezetében.

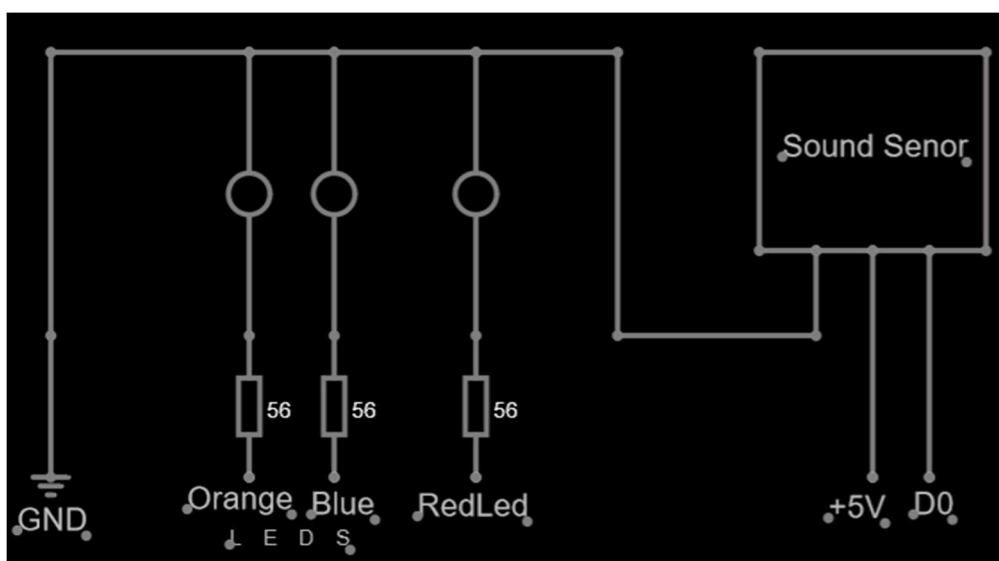
Használat során először össze kell kapcsolni a szoftvert és a hardvert, amit az alkalmazásban tehetünk meg. Összekapcsolódáskor speciális azonosítást használtam, tehát a szoftver elküld egy azonosító kérelmet a hardver felé, és ha a megfelelő választ kapja, az azt jelenti hogy megfelelő eszközt talált és kapcsolódik. Csak sikeres kapcsolódás után lehet elérni a további vezérléseket a programban, ezzel elkerülve a hamis vezérlést. Kapcsolódás után bekapcsolhatjuk az eszközt, amely ezt a ledet állapotának változtatásával jelzi is, egyben ezzel be is kapcsoljuk a szenzort, és a hang feldolgozásáért és küldéséért felelős kód részt. bekapcsolás után az eszköz hang monitorozó módban van, tehát csak szimplán kijelzi számunkra a hangerőszinteket, más beavatkozás nem történik. Ha riasztóként szeretnénk használni az eszközt azt a riasztó-mód bekapcsoló gomb megnyomásával tehetjük meg, ekkor a készülék riasztó módban van, de a riasztás még nem lett élesítve. Ehhez egy megjelenő csúszkán be kell állítani azt a szintet, amit ha elér a hang szintje, a riasztás megkezdődik. Egy aktiváló gombbal véglegesíteni tudjuk a beállított értéket, ezzel aktiválva a riasztót. Ha az említett esemény bekövetkezik a készülék és a program egyaránt villogással jelzi azt, egészen addig, amíg nem deaktiváljuk. Ezt követően újabb riasztás beállítására van lehetőség, vagy visszatérhetünk a hang monitorozó állapotba. Lehetőségünk van bármikor lecsatlakozni az eszköztől, illetve ki is léphetünk a számítógépes programból, ez esetben a program magától elvégzi a lecsatlakozást. Ha menet közben kihúzzuk az eszközt, legközelebbi bedugásnál alapállapotba lép, és újbóli csatlakozás után ismét használható.

A továbbiakban a megvalósítás pontos módjait és eljárásait tárgyalom pontokba szedve.

2. Hardver bemutatása

Maga a hardver egy Arduino nano panelből és egy hozzá készített POD nevű (Project One Device – magyarul: projektmunka egy eszköz -) áramkörből áll. A POD egy különálló nyákon foglal helyet, melyen az áramkör fontosabb kivezetései megtalálhatóak, ezeket egy egyszerű szalagkábel köti össze az Arduino panellel így összeszerelése, és csatlakoztatása rendkívül egyszerű, gyors. Ezt a panelt ilyen módon bármilyen Arduino eszközhöz pillanatok alatt könnyedén hozzácsatolhatjuk, így tulajdonképpen egy mikrovezérlős perifériát alkot. A szalagkábel összesen 6 érből áll így elfér kevesebb mint egy porton, a 6 információ-kábel egymás mellett meghatározott sorrendben helyezkedik el így egy csonka byte-ot alkot. A kapcsolás így néz ki:

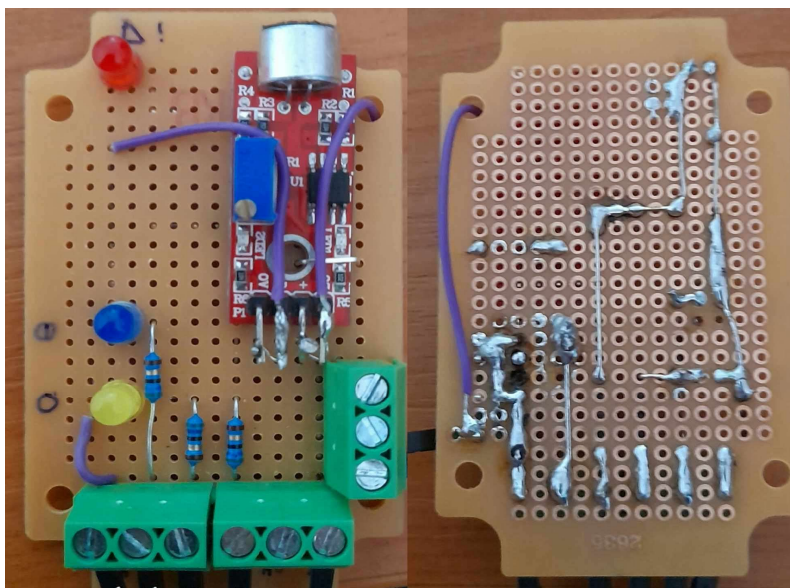
A POD kapcsolási rajza:



A kapcsolás egyszerű, három visszajelző ellenállást tartalmaz, és a hozzájuk tartozó előtét ellenállásokat, amelyek egy rövid kalkuláció után 56 ohmosnak lettek kiválasztva. Az alkatrészek közös földpontra vannak helyezve, illetve a hangérzékelő kivezetései találhatóak meg rajta. Ezek az Arduino D port lábaira vannak kötve. A kapcsolódást és a kikapcsolt állapotot a sárga színű led jelzi, a bekapcsolt állapotot a kék színű led. A riasztó mód bekapcsolását pedig a piros led jelzi. A hangérzékelő egy egyszerű KY037-es mikrofon modul, ami egy analóg és egy digitális kimenettel rendelkezik, a POD-on mindkettő kivezetés megtalálható, de használva jelenleg csak a digitális kimenet van. A kivezetések sorkapoccsal lettek megoldva, melyekbe egyszerűen elég becsavarozni a megfelelő kábeleket és kész is a hosszútávú bekötés.

Megvalósítását tekintve elég egyszerű, a projekt egy spórolósabb költségvetésből készült, így lyukacsos próba nyákon raktam össze. Az alkatrészek a fenti kapcsolási rajznak megfelelően helyesen be vannak forrasztva, így egy stabil áramkört kapunk eredményül. A kivitelezés a következő ábrán látható:

A POD fizikai megvalósítása:



A képen láthatjuk az eszköz kezdetlegességét, ugyanakkor teljesen működőképes. A továbbfejlesztés rovatban szó lesz ezen áramkör komolyabb megtervezése mellett a további hardveres és szoftveres fejlesztésekről.

3. Szoftver részegységeinek bemutatása

Következőnek nézzük meg a szoftver részegységeit. A projekt elkészítésében a programozásra volt szükség legnagyobb mértékben, ugyanis 3 különböző nyelvet kellett használni. A programozási feladat először is egy mikrovezérlő programból áll, melyet C-ben írtam, egy grafikus felületről, mely Design elemeit Xaml-ban, míg a logikai részét C#-ben készítettem el. Az alábbi pontokban ezeket a részegységeket veszem górcső alá.

3.1. Mikrovezérlő program.

A mikrovezérlő programja egy egyszerű C kódból áll, a D portra vannak kötve a kapcsolási rajzon látható kimenetek, és ezek kimenetként vannak beállítva a Setup kódrészben, kivéve a D0 csatlakozót, ami hangbemenet, tehát az bemenetként van inicializálva, az alkalmazás lényegében egy válaszfigyelő ami kommunikál a számítógépes alkalmazással és reagál annak utasításaira. A main program tehát egy egyszerű Switch-Case szerkezetből áll ami a válasz értékétől függően kezeli a mikrokontrollert (ledek le-fel kapcsolása, szenzor engedélyezése, stb...)

3.1.1. Soros kommunikáció

Az alkalmazás USB-n keresztül kommunikál a számítógéppel, tehát soros kommunikációt valósít meg. A kommunikáció 9600as frekvencián történik. A számítógépes alkalmazás kis illetve nagy karaktereket (betűket, a-tól D-ig) küld a mikrovezérlő felé, ami megvizsgálja a kapott választ és a szerint lép be az egyes esetekbe, és hajtja végre a kódot. Például, amikor csatlakoztatni akarjuk a szoftvert a POD-hoz, ki kell válasszuk a megfelelő soros portot(COM).

Ekkor az alkalmazás küld egy kis 'a' -karaktert a kiválasztott portra majd vár. Ha a kiválasztott porton ténylegesen egy POD van, akkor arra megérkezik az 'a' karakter és ennek a kezelése meg van írva a mikrovezérlő programjában. Amennyiben a soros porton érkező válasz 'a' karakter, a mikrovezérlő felkapcsolja a sárga lámpát, és küld egy „POD1” nevű azonosító üzenetet vissza a soros porton, ezzel jelezve a számítógép felé, hogy a találat sikeres volt. Ekkor megjelenik a „Connected” felirat az alkalmazásban, és elérhetővé válik a POD további vezérlése. Ha bekapcsoljuk az eszközt, akkor aktiválódik a hangszenzor jelének feldolgozásáért felelős kódrészlet, és a hanginformáció folyamatos áramlása megkezdődik a soros porton az alkalmazás felé.

3.1.2. Hangérzékelő egység.

A hangérzékelő egység nagyon gyorsan, milliszekundumonként több információt küld. A digitális kimenet úgy működik, hogy beállítunk a hangérzékelőn található potméterrel egy küszöbértéket, és ha az alá megy a jelszi kisebb snt 0-át, ha a fölé, akkor 1-et küld. Tehát kulcsfontosságú ezt a szintet megfelelően beállítani, attól függően, hogy mennyire szeretnénk, hogy érzékeny legyen a mikrofon. Ha csendhelyzet van, a szenzor folyamatosan 0 információkat küld, hang esetén pedig 1-eseket. Hogy a zajnak a szintjét is meg tudjuk mérni ennél egy kicsit többre van szükség. Beállítunk egy mintavételező frekvenciát, jelen esetben 10 milliszekundumot, és adott időközönként megvizsgáljuk, hogy még mindig tart-e a beszéd, és 1-es értékeket kapunk vissza, vagy már vége van és 0-s értékek érkeztek. Ameddig egybefüggően 1-eseket kapunk vissza, addig egy számlálót növelünk. Amennyiben 0-át észlelünk a számlálót nullázzuk. Tehát minél tovább tart egybefüggően az észlelt hang időtartama, annál nagyobb értékű a számlálónk. Ez az érték feleltethető meg az amplitúdónak, mert minél nagyobb elánnal beszélünk, annál kisebb az esélye, hogy a küszöbérték alá csökken az amplitúdó, ezért nagyobb értékeket is elér a számláló. Halkabb beszéd esetén pedig a kis megszakadásoknál, zöngétlen betűknél a küszöbérték alá csökkenhet az amplitúdó ami a számláló nullázását, ezáltal kisebb számláló értékeket eredményez. Amikor az észlelt hangszint végig a küszöbérték alatt van, nullát vesz fel folyamatosan a számláló, így jól megtudjuk különböztetni mikor van csend, és mikor nincs. Amennyiben be van kapcsolva az eszköz, ezeket az értékeket folyamatosan elküldjük az alkalmazás felé, ami feldolgozza azokat. Annak érdekében hogy az alkalmazásban könnyebb dolgunk legyen, egy „map” függvénnyel az értékek nagyságának függvényében átskálázzuk az értékeket úgy, hogy azok 0-éa 100 közé essenek, így a hangerőt egy 0 és 100 közötti skálán feldolgozva küldjük tovább az alkalmazásnak.

3.1.3. Egyéb kódrészek.

Az eddigieket leszámítva, már csak egy pár kódrész van, mint például a szenzor ki bekapcsolásáért felelős enableSensor(0/1) függvény, aminek az argumentumába engedélyezni, illetve tiltani tudjuk a hangérzékelőt. Ennek a függvénynek visszatérési értéke is van, attól függően, hogy ki vagy be van-e kapcsolva a szenzor, így bármikor ellenőrizhetjük. A függvény továbbá egy belső változót is állít igaz/hamis értékre, amit a hangérzékelő függvényébe beépített számláló is

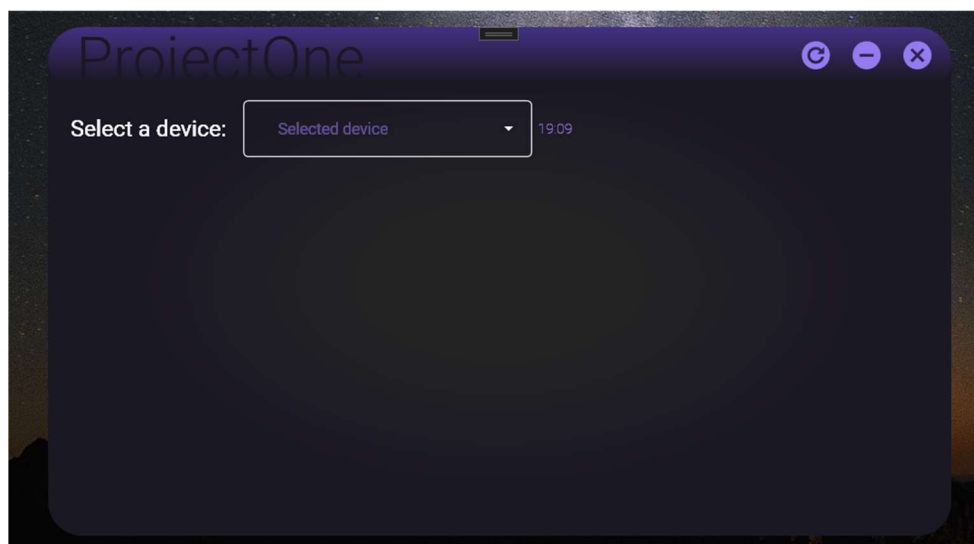
ellenőriz, és ha használva a függvényt, kikapcsoljuk a szenzort, akkor a belső változó hamis értéket vesz fel, ezáltal a hanginformáció küldése is megszakad, hogy redundáns null információk küldésével ne terheljük le a sorosportot.

3.2. Számítógépes program

Az alkalmazás két főrészből épül fel: a grafikus felületből ami megjeleníti, megkönnyíti a felhasználó számára a kezelést, és a mögötte lévő kódot, ami működővé teszi a felületet, és kommunikál az eszközzel. Az alkalmazás a .NET keretrendszerben készült, így a legtöbb platformhoz kompatibilis.

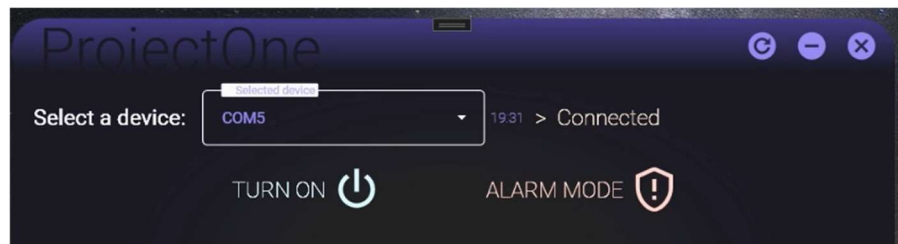
3.2.1. Kezelői felület

A kezelői felület letisztult sötét témában jelenik meg előttünk, kezelését tekintve, kevés, de nagy ikonok szolgálnak a vezérlésre, melyek egy kényelmes felhasználói élményt nyújtanak. Megnyitáskor az alábbi formában jelenik meg az alkalmazás:

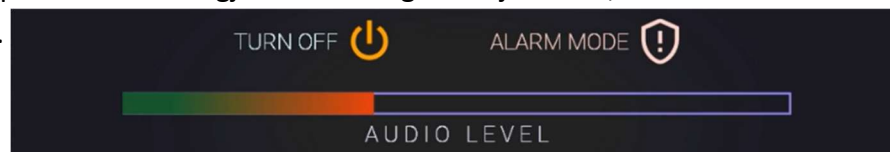


Itt egy úgynevezett „ComboBox” -ból, tehát egy legördülő listából tudjuk kiválasztani a számítógépen épp elérhető sorosportokat. Lehetőségünk van a frissítés ikonra kattintva ezt újra tölteni, ami nem csak a listát frissíti, de a kezelői felületet is visszaállítja alaphelyzetbe. Továbbá minimalizálni tudjuk az ablak méretét, illetve bezárni az alkalmazást. Természetesen mind a frissítés, mind a bezárás lecsatlakoztatja az eszközt az alkalmazásról.

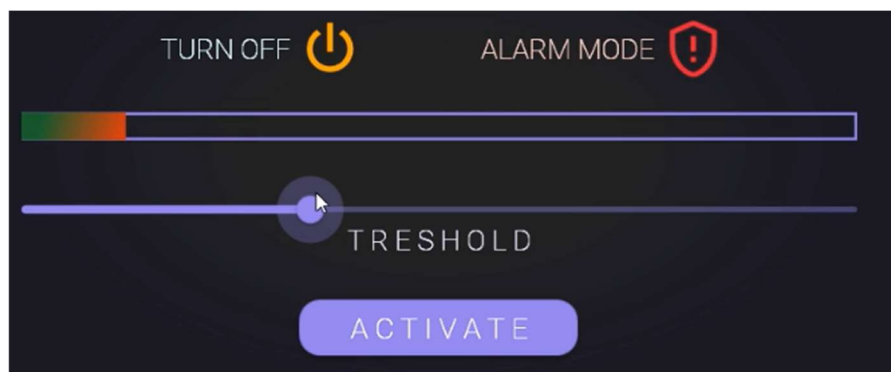
A sikeres kapcsolódás után megjelennek a további vezérlési lehetőségek gombjai a felületen, és a választó mellett a csatlakozás időpontja:



Itt lehetőségünk van bekapcsolni az eszközt, és a riasztó módot. Természetesen az utóbbit csak akkor tudjuk aktiválni, ha az eszköz be van kapcsolva, és ha kikapcsoljuk az eszközt, a riasztó mód is automatikusan kikapcsolásra kerül. Bekapcsolás után megjelenik a hang visszajelző sáv, és elérhetővé válik a riasztó mód.

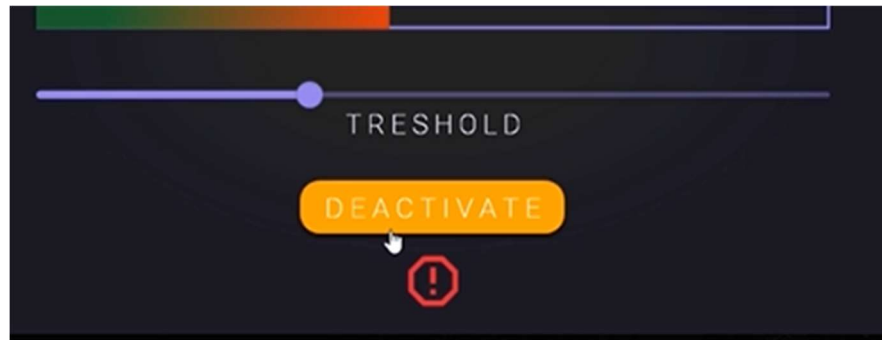


A riasztó mód aktiválása esetén megjelenik a riasztás szintjét beállító csúszka és az aktiváló gomb.



Az aktiváló gomb, annak érdekében, hogy ne lehessen nagyon minimális értékű riasztási küszöböt beállítani csak 30-as, vagy annál nagyobb érték fölött jelenik meg, értékképpen a csúszka teljes tartománya 100egység, tehát ha a csúszka közel egyharmadánál van, az aktiváló gomb megjelenik, és bármilyen a feletti érték beállítható. Ha a csúszkát a beállított szint alá visszük, a gomb eltűnik.

Aktiváláskor a csúszka aktuális értéke elmentődik, és egy időközönként lefutó programrész folyamatosan figyeli, hogy az érkező hanginformáció (0 és 100 közötti szám) nagyobb e mint a beállított érték (30 és 100 közötti szám). Mivel a csúszka és a hangkijelző skálázása megegyezik, ezért ahova a csúszkát húzzuk, az lesz az a pont is, ami fölé ha a hangcsík megy, megtörténik a riasztás, tehát remekül vizualizálja a felület a működést. A riasztó persze csak az aktiváló gomb megnyomása után lesz éles, ha ezután megtörténik az esemény, a riasztás bekövetkezik. A következő képen megfigyelhető egy ilyen esemény.



A képen láthatjuk, hogy a hangszint a csúszka értéke fölé ment, és megjelent a felkiáltójel villogása, egyúttal, a POD-on a piros led villogása, ami a riasztást jelzi. Ez egészen addig folyamatosan villog, amíg a Deaktiválás gombjára rá nem kattintunk. Ekkor eltűnik a felkiáltójel, és a villogás megszűnik.

Igyekeztem a felhasználói felületet ergonomikusra tervezni, tehát kevés de egyértelmű irányítást elhelyezni, azokat pedig formalitásában (név,szín,ikon) a hozzá kötődő funkcióval valamilyen párhuzamba vonni, ami által egy intuitív, könnyen kezelhető és olvasható felületet kapunk. Hogy ezt a letisztultságot növelni tudjam, a MaterialDesign könyvtárát hozzáadtam a projekthez, ami egy ingyenes könyvtár, és olyan cégek használják mint pl. a Google. Ez további elemeket ad hozzá a forráshoz, mint például a hangvisszajelzőt, a csúszkát, a letisztult gombokat és az ikonokat.

3.2.2. Alkalmazás működése

Az alkalmazás nem más mint egy függvények halmazából álló nagy C# kód. Vannak „események” amik ha bekövetkeznek van egy saját kódrészlet, ami lefut abban az esetben. Például ha egy gombot megnyomok, a megnyomás egy eseménynek minősül, ami lefuttatja a gomb megnyomás-függvényét és valami történik. Hogy egy konkrét példát is mutassak, nézzük meg a bezáró gomb függvényét:

```
private async void btClose_Click(object sender, RoutedEventArgs e)
{
    if (spl != null && spl.IsOpen)
    {
        try
        {
            spl.Write("b");
            spl.Write("a");
            await Task.Delay(100);
            spl.DiscardInBuffer();
            spl.Close();
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }

    Application.Current.Shutdown();
} // bezaras
```

A bezáró gomb egy objektum aminek a tulajdon neve „btClose”. Ennek a gombnak az eseménykezelőjéhez hozzá van adva a „btClose_Click” nevű esemény, tehát ha a gombra kattintok, ez a függvény végrehajtódik a képen látható módon, vagyis megnézi, hogy van-e megnyitott sorosport kapcsolat. Ha van, és a bezárásra kattintunk az azt jelenti, hogy be kell zárni a portot, és lecsatlakozni, mielőtt kilépünk. Ennek érdekében egy try-catch művelet megpróbálja elküldeni az 'a' és 'b' karaktereket a mikrovezérlőnek (amelyek a ledek lekapcsolását és a szenzor letiltását jelentik) vár 100 milliszekundumot hogy végrehajtsa a művelet, majd kiüríti a sorosport buffert, aztán bezárja a portot. Ha nem sikerült a portot elérni egy automatikus hibaüzenet jelentkezik. Miután mindezeket elvégeztük, az „Application.Current.Shutdown()” kódrészlet teljesen bezárja az alkalmazást.

Ez csak egy kis egyszerű példa, a program ilyen függvények tömkelegéből áll. Az egyik legfontosabb ilyen függvény, az pont egy saját függvény, ami a listaválasztó kezelőfüggvényéből hívódik meg, tehát minden alkalommal, amikor kiválasztunk a legördülő listából egy portot, azon a porton megpróbál kapcsolatot létrehozni egy POD eszközzel. Ezt úgy teszi, hogy létrehozza az alkalmazásban is megnyitja a soros portot 9600-as frekvencián, és az általunk kiválasztott portra elküldi az azonosítás kérelmet. Ha érkezik válasz, és az megfelelő („POD1”) akkor létrejön a kapcsolat, a beállított portra elküldi azokat a karaktereket, amik az eszköz sárga ledjének felkapcsolásáért felelnek (ez jelzi, hogy a készülék kapcsolódott, de kikapcsolt állapotban van) illetve a kiválasztott sorosport eseménykezelőjéhez hozzáadódik egy függvény, ami akkor fut le, ha egy adat érkezik a soros porton. Erre azért van szükség, mert ha bekapcsoljuk az eszközt, az folyamatosan küldeni fogja az információkat, amiket gyorsan kell feldolgozzunk, így ha bármilyen adat érkezik, az adatok feldolgozásáért felelős függvény lefut.

3.2.3. Beérkező hangadat feldolgozása

Mint már korábban tárgyaltuk, a mikrovezérlőből egy 0 és 100 közötti érték jön be az alkalmazásba. Ahhoz hogy megtudjuk különböztetni a beáramló számok közül az új értékeket, egy L betűt küldünk minden új adat előtt (az L a Level szóból ered jelen esetben) tehát pl a 8435528978 számsorozat így jelenik meg: L84L35L52L89L78 ezáltal könnyen komponensekre bontható az adatstream: 84; 35; 52; 89; 78. Az adatfeldolgozó függvény tehát L betűket keres, és az utánuk érkező adatot kiolvassa, és elmenti egy változóba, ilyen módon bekerül a programba a POD-ból érkező hanginformáció. A hangérzékelő szenzor nagyon gyors és előfordulhat, hogy egymás után egymástól nagyon eltérő jelszintek jelennek meg, ami a vizualizációban elég zavaró lenne, mert nagyon gyorsan

ugrálna a hang visszajelző csík, amit egyrészt nehéz feldolgozni, másrészt meg úgy néz ki, mintha villogna, ami igencsak kellemetlen hatást kelt a felhasználóban.

Annak érdekében, hogy ezt ki küszöböljük egy átlagoló szűrőt raktam a beérkező adatokra. Ez a szűrő egy objektumnak felel meg, aminek van egy „ablakméret” tulajdonsága, ami azt jelenti, hogy az utolsó hány darab információt átlagolja, magyarul hogy mekkora legyen a filter ablaka. Ha ezt megadtuk, akkor a beérkező információkat egy átmeneti listára szedi, majd a beérkező értékek összegét elosztja a listán szereplő adatok darabszámával, magyarul átlagot von. Jelen esetben 8 az ablak méret, tehát minden 8. beérkező információ után lesz egy átlagolt információ. Ez az átlagolt információ kerül végül megjelenítésre, így sokkal kevésbé fog ugrálni, villogni, és sokkal finomabban fog a hangvisszajelző sáv változni. Ezzel a szűrővel leszűrhetjük a hirtelen megjelenő magas impulzusokat, amivel elkerülhetjük a hamis riasztást. Ehhez az átlagolt értékhez viszonyítjuk a riasztási küszöbértéket is, így csak akkor fog beriasztani a rendszer, ha a hangerőszint ténylegesen (legalább egymás után 8 alkalommal meghaladta átlagosan) a riasztási szintet, a tüskék pedig eltűnnek.

4. Továbbfejlesztési lehetőségek

Mivel egy kezdetleges projektről van szó, így fontos beszélni a továbbfejlesztési lehetőségekről, mert van sok. Talán a legelső és legfontosabb fejlesztés az a nyák és opcionálisan a ház megtervezése lenne, amivel a projekt kilépne a béta szintről és ténylegesen sorozatgyártható termékként funkcionálna. A saját nyák lehetőséget adna rá, hogy sokkal több funkciót belevigyünk, sokkal kompaktabb megvalósítás mellett. Például hozzáadhatunk több ledet, vagy rgb ledet, kijelzőt, amivel szövegesen is tájékozódhatunk, csipogót ami hangjelzést ad gombok megnyomásakor, aktiváláskor vagy riasztáskor, és hozzáadhatnánk pár gombot is, hogy alkalmazás nélkül is lehessen pár funkciót elérni, egy afféle „karbantartó” módban. SMD alkatrészek bevonásával pedig sokkal kisebbre, kompaktabbra meg tudjuk csinálni, ami nem utolsó szempont egy biztonságtechnikai berendezésnél, mert minél kisebb, annál könnyebb elrejteni/nehezebb megtalálni, és annál nehezebben hozzáférhető. A ház létrehozása az esztétikai tényezőkön kívül megvédheti az áramkört környezeti tényezőktől, például a víztől vagy fizikai erőhatásoktól.

Akusztikai oldalról nézve úgy lehetne tovább vinni a projektet ha egyrészt, megválogatnánk a mikrofont, például a jelenlegi mikrofon elég rossz karakterisztikával bír, és kis környezetben vesz fel, de ha olyan mikrofont használunk ami sokkal nagyobb térben rögzít, szélesebb tartományban ezáltal effektívebben véd, másrészt, ha több kis mikrofon modult készítenénk, amik egymással kommunikálnak, és ezt több különböző helyen szétszórni, pl.: ajtók fölé, fontos helységekre ezáltal a figyelésünk lefedettsége kiterjedne térben, és frekvenciában ami sokkal pontosabb eredményt adna.

Továbbá ki lehetne egészíteni egyéb szenzorokkal, amiket mint külön komponenseket meg lehetne vásárolni a többihez kiegészítőként és ugyanúgy rákapcsolódna a vezérlő rendszerre, így módon nem csak hangot hanem pl.: hő, mozgást, fényvisszaverődést, vagy akár elektromágneses hullámokat is érzékelhetnénk amivel egy egészen személyre szabható biztonsági rendszer alakulna ki.

Végül soron pedig valamilyen WiFi egyéget a vezérlő rendszerbe iktatva, elérhető, hogy mind a készülék, mind az alkalmazás csatlakozzon az internethez, és ha riasztás van például egy Android telefonos alkalmazásra értesítést küldjön amennyiben a felhasználó magánszemély, vagy értesítse a hatóságokat, amennyiben a felhasználó egy cég.

Természetesen számos irányba fejleszthető még a projekt, csak azokat a szempontokat soroltam fel, amely vízióim már a projekt készítése közben is megvoltak csak bármilyen tényező (anyagi, idő, stb..) miatt a megvalósításuk már nem lett volna kivitelezhető.

5. Végszó

Köszönöm, hogy elolvasta a projektbeszámolómat, remélem ismertetésem elég kielégítő és figyelemfelkeltő volt. Hogy még jobb betekintést nyerhessünk a projektem működésébe, készítettem egy rövid videós bemutatót, ami az alábbi youtube-linken megtekinthető: <https://youtu.be/YGv0JO9AqMY>

2023.12.15, Budapest.

OE-KVK-HTT

Garai Maja